TD1 — Terminal & Git fundamentals

Git, Python Environments & Coding Best Practices

MSA-DATB07-03 — 1 h 30 practical

Learning outcome. Install the tools, read and search files from the terminal, then create a Git repository, save changes with `git add` and `git commit`, read a history with `git log`, and undo mistakes with `git restore` and `git revert`.

Tip — how this TD works

Most questions give you a command. For each one: **1. predict** what it will do, **2. run** it, **3. explain** the result and each part of the command. Write your answers in `td1_answers.md` (exercise 3 tells you where). A few questions marked **Your turn** ask you to write a command yourself. The slides and the bash cheat-sheet are on https://github.com/msa-datb07-git: keep them open.

Part 0 — Install and set up *(20 min)*

Exercise 1 — Install Git, Python and VS Code

Follow the column of your system. If one of the tools is already installed, check its version and skip the step.

Windows

1. **VS Code**: https://code.visualstudio.com, default options.
2. **Git for Windows**: https://git-scm.com. Keep the defaults, except two screens: *default editor* → "Use Visual Studio Code"; *name of the initial branch* → "Override" and type `main`. Git for Windows installs Git *and* Git Bash, but **not** Python.
3. **Python 3.10 or newer**: https://www.python.org/downloads. On the first screen, tick **Add python.exe to PATH**.
4. In VS Code: Extensions (`Ctrl+Shift+X`) → install *Python* (Microsoft). Then `Ctrl+Shift+P` → *Terminal: Select Default Profile* → *Git Bash*.

macOS

1. Open *Terminal* (Spotlight: `Cmd+Space`, type "terminal"). Type `git --version`. If macOS offers to install the *command line developer tools*, accept: this installs Git.
2. **Python 3.10 or newer**: https://www.python.org/downloads, run the `.pkg` installer.
3. **VS Code**: https://code.visualstudio.com, drag it into *Applications*.
4. In VS Code: Extensions (`Cmd+Shift+X`) → install *Python* (Microsoft). Then `Cmd+Shift+P` → *Shell Command: Install 'code' command in PATH*.

Close VS Code completely, reopen it, and open a terminal with `Ctrl+'` (the key left of 1). Check everything:

```
echo $0            # bash (Windows, Git Bash) or zsh / -zsh (macOS)
git --version      # git version 2.x
python --version   # Python 3.10 or newer   (macOS: python3 --version)
code --version     # the VS Code version
```

Pitfall — python opens the Microsoft Store, or says "not found"

Windows has a fake `python` that sends you to the Store. Fix: Windows *Settings* → *Apps* → *Advanced app settings* → *App execution aliases*, turn off the two *python* entries, then reopen VS Code. If it still fails, run the Python installer again and tick *Add python.exe to PATH*.

Exercise 2 — Tell Git who you are

Once per computer, type these four commands with your own name and e-mail (the e-mail you will use for GitHub in session 3):

```
git config --global user.name  "Firstname Lastname"
git config --global user.email "firstname.lastname@student.example.edu"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

- `git config` changes a Git setting; `--global` means "for every repository on this computer".
- `user.name` and `user.email` are written on every commit you make, so that your teammates know who did what.
- `init.defaultBranch main`: every new repository starts with one line of work, called a *branch*. This gives it the name `main`. Older Git versions call it `master`; with this setting everybody has the same name.
- `core.editor "code --wait"`: sometimes Git needs you to write a text, for example a message. It then opens VS Code (`code`) and waits (`--wait`) until you save and close the tab before continuing.

Check with `git config --global --list`: you should see the four values.

Exercise 3 — Your course folder, and the TD material

1. Create a folder for this course inside *Documents*, and go into it:

   ```
   cd ~/Documents           # ~ = your home folder
   mkdir -p courses/git     # create courses, and git inside it (-p: the whole chain at
        once)
   cd courses/git
   pwd                      # check where you are
   ```

   If `cd ~/Documents` answers "No such file or directory" (common on Windows when Documents is in OneDrive), type `cd` followed by a space, drag your *Documents* folder from the file explorer onto the terminal, and press `Enter`. Then continue with `mkdir`.

2. Create your answers file *here*, in `courses/git`, next to the repositories you will download, not inside them: `code td1_answers.md`.

3. Download the TD material. `git clone <address>` downloads a copy of a project *with its whole history* and creates a folder with its name:

   ```
   git clone https://github.com/msa-datb07-git/td1-shell-and-git.git
   cd td1-shell-and-git
   ls
   ```

Pitfall — CRYPT_E_NO_REVOCATION_CHECK on Windows

Some networks (company, campus) block a certificate check that Windows performs. If `git clone` or `curl` fails with this message, run `git config --global http.schannelCheckRevoke false` for Git, and add `--ssl-no-revoke` to `curl` commands.

Part A — Explore with the terminal *(30 min)*

Everything happens in the **terminal of VS Code** (Git Bash on Windows, zsh on macOS): no file explorer, no mouse. Start by going into the practice folder: `cd shell-lab`.

Exercise 4 — Look around

Predict, run, explain:

1. `pwd`
2. `ls`, then `ls -la`. Which names appear only with `-la`? What do they have in common?
3. `ls -la .secrets`, then `cat .secrets/note.txt`
4. `cat README.txt`

Your turn: go into `docs/archive`, list what is there, then come back to `shell-lab` in one command.

Exercise 5 — Read the data

Predict, run, explain:

1. `head -n 3 data/raw/sales_2024_q1.csv`. What are the columns?
2. `wc -l data/raw/sales_2024_q1.csv`. The answer is 41: how many *sales* does the file contain, and why not 41?
3. `wc -l data/raw/*.csv`. What does `*` do here? Which quarter has the most sales?

Your turn: show the last 2 lines of the fourth-quarter file.

Exercise 6 — Search

Predict, run, explain:

1. `grep -r PENGUIN .` — which file contains the word, and what does the `.` mean?
2. `grep -rl PENGUIN .` — what changes with `-l`?
3. `find .  -name "*.md"`

Your turn: list every CSV file of the folder with `find`.

Exercise 7 — Send results to a file, chain commands

Predict, run, explain:

1. `grep -c ERROR logs/server.log`
2. `grep ERROR logs/server.log | head -n 3` — what does `|` do?
3. `grep ERROR logs/server.log > errors.txt`, then `wc -l errors.txt`
4. `grep WARN logs/server.log >> errors.txt`, then `wc -l errors.txt` again. Why is the number bigger? What would have happened with `>` instead of `>>`?

Exercise 8 — Your computer and the network

Predict, run, explain. When a command differs between Windows and macOS, both are given.

1. `echo $HOME`, then `which python` (macOS: `which python3`)
2. `ping github.com` (Windows) or `ping -c 4 github.com` (macOS). Did all the test messages come back? What is the average time?
3. `nslookup github.com`: what is the IP address of `github.com`?
4. `curl -I https://github.com`: what is the status code, and what does it mean?
5. The current temperature in Paris:

```
curl -s "https://api.open-meteo.com/v1/forecast?latitude=48.85&longitude=2.35&
    current=temperature_2m" | python -m json.tool
```

Find the temperature in the answer. Why are the quotes around the address needed?

Part B — Your first repository *(35 min)*

Exercise 9 — Six commits, one project

Go back to your course folder: `cd ../..` (you are now in `courses/git`). You build your own project from scratch.

Guided: the first two commits. Run each command, and explain what it did.

```
mkdir my_sales
cd my_sales
git init                                    # turn this folder into a Git repository
git status                                  # what does Git say?
code README.md                              # write one line: # My sales analysis, save
git status                                  # what changed?
git add README.md
git status                                  # and now?
git commit -m "Add project README"
git log --oneline
mkdir data
cp ../td1-shell-and-git/project/data/sales.csv data/
git add data
git commit -m "Add Q1 2024 sales data"
git log --oneline
```

1. What does `git status` say about `README.md` before `git add`, after `git add`, and after `git commit`?
2. How many lines does `git log --oneline` show at the end, and what is on each line?

Your turn: the next commits, each with `git add` then `git commit -m "..."` and a clear message.

3. Create `analysis.py` with this function, then commit it:

```
import csv


def load_sales(path):
    """Read a sales CSV file and return a list of dictionaries (one per row)."""
    with open(path, newline="") as f:
        return list(csv.DictReader(f))
```

4. Add this second function at the end of `analysis.py`, save, and *before committing* run `git diff`, then `git add analysis.py`, then `git diff` again, then `git diff --staged`. What does each of the three show? Then commit.

```
def total_revenue(rows):
    """Sum quantity * unit_price over all rows."""
    return sum(float(r["quantity"]) * float(r["unit_price"]) for r in rows)
```

5. Create a file `.gitignore` containing these three lines, then commit it:

```
__pycache__/
outputs/
.env
```

6. Create a file `.env` containing `API_KEY=secret`, and run `git status`. Is `.env` listed? Why?
7. Show the history: `git log --oneline`, then `git log --stat`. What does `--stat` add?

Exercise 10 — Read someone else's history

Go to the downloaded repository: `cd ../td1-shell-and-git`. Predict, run, explain:

1. `git log --oneline`: how many commits?
2. `git log`: who made the third commit from the top, and when?
3. `git log --stat`: one commit added a file that should never have been committed. Which commit, which file? (Press `q` to quit the list.)
4. `git show da319b1`: what is in that file? Why is it a problem?

Exercise 11 — Undo

Still in `td1-shell-and-git`. If `git status` lists files you created in Part A (`shell-lab/errors.txt`), ignore them: they do not affect what follows.

1. **Staged too early.** Add a line to `README.md` and save. Then predict, run, explain:

```
git add README.md
git status
git restore --staged README.md     # is the line still in the file?
git status
git restore README.md              # and now?
```

2. **A bad commit: revert it.** Predict, run, explain:

```
git revert da319b1     # VS Code opens with a message: keep it, save, close the tab
git log --oneline
ls project
```

   Is `config.ini` still in the folder? Is it still in the history (`git show da319b1`)?

3. **The exception: a commit you just made.** Add a line to `README.md`, then:

```
git add README.md
git commit -m "oops"
git reset --soft HEAD~1      # HEAD~1 = the commit before the current one
git status                   # where is your change now?
git commit -m "Add a line to the README"
git log --oneline
```

4. In one sentence: why is `git revert` the default way to undo a commit, and `git reset` only an exception?

Part C — Checklist *(5 min)*

1. In Git Bash, what does `echo $0` print? What does it mean if it prints something else?
2. What does `Ctrl+C` do in a terminal? How do you copy text instead?
3. What is the difference between `>` and `>>`?
4. Show your `my_sales` history to the instructor: `git log --oneline`.

Deliverable

Before leaving: `td1_answers.md` filled in, in your `courses/git` folder; the `my_sales` repository with at least 5 commits; the three undo scenarios done in `td1-shell-and-git`. Nothing to send today: session 3 puts your work on GitHub.